MicroGrad: A Scalar Automatic Differentiation Engine

Mariam Riyad Freeg
Student ID: 23P0147
CSE485–CSE488 Deep Learning

October 2026

Abstract

This report presents a scalar automatic differentiation engine and neural network API implemented without PyTorch autograd. Arithmetic operations build a computational graph, and reverse-mode backpropagation calculates gradients using the chain rule. The implementation passed all 12 course template tests. A multilayer perceptron trained on 21 synthetic examples reduced its recorded mean squared error from 0.643510302 to 0.002372670 in 200 epochs.

1 Introduction

Automatic differentiation supplies the gradients needed to train neural networks. This assignment implements that mechanism using individual scalar values. Each result remembers its operands and a local derivative rule. A reverse traversal combines those rules to obtain gradients of a final output with respect to earlier values and model parameters. The design is inspired by MicroGrad and follows the course assignment.

2 Mathematical Background

For an operation $z = f(a, b)$ and a final loss L, the chain rule is

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial a}.$$

The implemented local derivatives are

$$\begin{aligned}
z = a + b: &\quad \frac{\partial z}{\partial a} = 1, \quad \frac{\partial z}{\partial b} = 1,\\
z = ab: &\quad \frac{\partial z}{\partial a} = b, \quad \frac{\partial z}{\partial b} = a,\\
z = a^n: &\quad \frac{\partial z}{\partial a} = na^{n-1},\\
z = \tanh(a): &\quad \frac{\partial z}{\partial a} = 1 - \tanh^2(a),\\
z = e^a: &\quad \frac{\partial z}{\partial a} = e^a.
\end{aligned}$$

Worked numerical example

Let $a = 3$, $c = a + a$, and $d = ca$. The forward values are $c = 6$ and $d = 18$. Backpropagation starts with $\partial d/\partial d = 1$. Multiplication contributes

$$\frac{\partial d}{\partial c} = a = 3, \qquad \left.\frac{\partial d}{\partial a}\right|_{\text{direct}} = c = 6.$$

Addition uses a twice, contributing 3 from each input. The accumulated gradient is therefore

$$\frac{\partial d}{\partial a} = 6 + 3 + 3 = 12.$$

The result also follows from $d = 2a^2$ and $\partial d/\partial a = 4a = 12$ at $a = 3$.

3 Computational Graph and Backpropagation

A `Value` output stores references to its operands. Figure 1 shows the worked example. Solid blue arrows represent forward computation. Dashed red arrows show the reverse direction of gradient flow. A bar over a variable denotes its accumulated gradient.

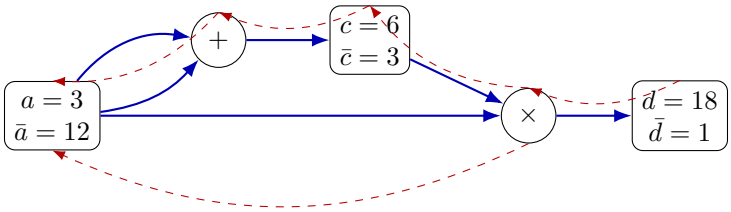


Figure 1: Forward values and reverse gradients for $d = (a + a)a$ at $a = 3$.

Each operation attaches a local `_backward` function. The global `backward()` method seeds the output gradient with 1 and invokes those local functions in the correct order.

4 Topological Sorting Algorithm

A node must receive every downstream contribution before its local backward rule runs. Depth-first search constructs a topological list:

```
topo = []
visited = set()

def build_topo(node):
    if node in visited:
        return
    visited.add(node)
    for parent in node._prev:
        build_topo(parent)
    topo.append(node)

build_topo(output)
output.grad = 1.0
for node in reversed(topo):
    node._backward()
```

The algorithm appends each node *after* visiting its parents. Reversing the resulting list processes the output before its dependencies. The visited set prevents a shared node from appearing twice in the list. Its gradient still accumulates contributions from every path.

5 Implementation of `Value`, `Neuron`, `Layer`, and `MLP`

`Value`

The `Value` class stores scalar `data`, accumulated `grad`, operand references in `_prev`, an operation label, and a local `_backward` function. Python arithmetic special methods create graph nodes for operations such as addition and multiplication. The implementation also supports numeric constants, reverse operations, subtraction, division, powers, `tanh`, and `exp`. Invalid operand types raise errors. `reset_grad()` clears one stored gradient; `_get_topo()` and `backward()` implement the global reverse pass.

`Neuron`

A neuron stores one `Value` weight for each input and one `Value` bias. Each is initialized with `random.uniform(-1, 1)`. Its forward method checks the input count and computes

$$h = \tanh\left(\sum_{i=1}^{m} w_i x_i + b\right).$$

`parameters()` returns its weights and bias; `zero_grad()` clears their gradients.

`Layer` and `MLP`

A layer applies its neurons to the same input. An MLP constructs successive layers and passes each layer's output to the next. Their `parameters()` methods flatten parameter lists, while their `zero_grad()` methods clear all parameter gradients.

6 Implementation and Validation

The course template tests were run from the project root with `py -m pytest -q`. The observed result was `12 passed`: nine engine tests and three neural network tests. The suite covers arithmetic, type errors, powers, activations, division by zero, graph gradients, network construction, and backpropagation.

Table 1: Analytical gradients compared with actual engine output.

Check	Expected	Observed
$a + a$ at $a = 3$: gradient at a	2	2
$(a + a)a$ at $a = 3$: gradient at a	12	12
$(ab)(a + b)$ at $a = -2, b = 3$: gradient at a	-3	-3
$(ab)(a + b)$ at $a = -2, b = 3$: gradient at b	-8	-8

A further numerical check used

$$F(x, y) = \frac{\tanh(xy + x^2) + e^y}{2}$$

at $x = 1.2$ and $y = -0.7$. Central finite differences used $h = 10^{-6}$; for example,

$$\frac{\partial F}{\partial x} \approx \frac{F(x + h, y) - F(x - h, y)}{2h}.$$

These comparisons check the derivative rules independently of the course test assertions.

Table 2: Autograd and finite-difference gradients.

Variable	Autograd	Finite difference
x	0.604841098199	0.604841098151
y	0.675239309448	0.675239309522

7 Experimental Setup

The saved notebook generated 21 inputs from -1.0 to 1.0 in increments of 0.1. Targets were

$$y = \tanh(1.5x).$$

This is a generated synthetic dataset, not an externally loaded file. The notebook fixed the random seed at 42 and constructed an MLP with one input, four hidden neurons, and one output. It has $4(1+1)+1(4+1) = 13$ trainable parameters.

Training used 200 epochs, learning rate $\eta = 0.05$, and full-batch mean squared error:

$$L = \frac{1}{N}\sum_{j=1}^{N}(\hat{y}_j - y_j)^2.$$

Each epoch cleared parameter gradients, calculated predictions and loss, called `backward()`, and updated parameters:

$$p \leftarrow p - \eta\frac{\partial L}{\partial p}.$$

8 Results and Visualizations

A fresh run reproduced the saved notebook's first and last recorded losses: 0.643510302 and 0.002372670. This is a reduction of about 99.63%. Figures 2 and 3 plot values from that reproducible run.

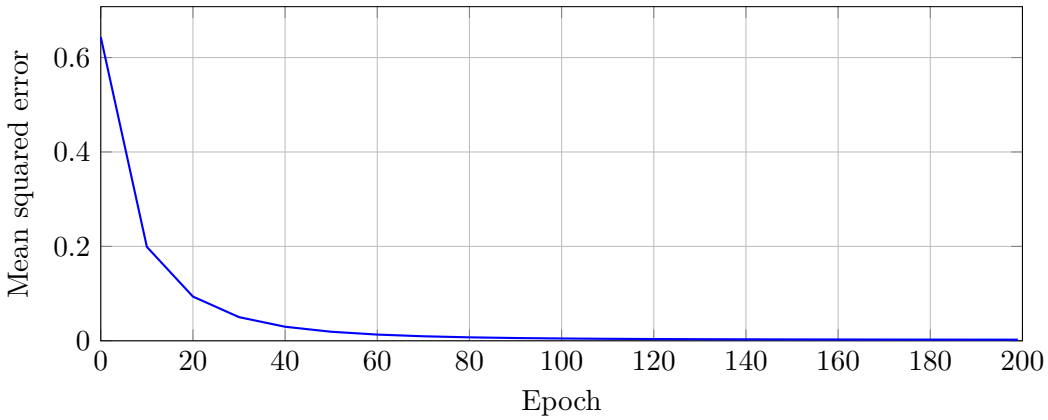


Figure 2: Training loss. Points are sampled every ten epochs and at the final recorded epoch from the notebook-equivalent run.

The trained curve is substantially closer to the target curve. Remaining error is visible near the positive endpoint: at $x = 1$, the target is approximately 0.905 and the final prediction is approximately 0.798, an absolute error of about 0.107. These results describe training examples only; no held-out test set was used.

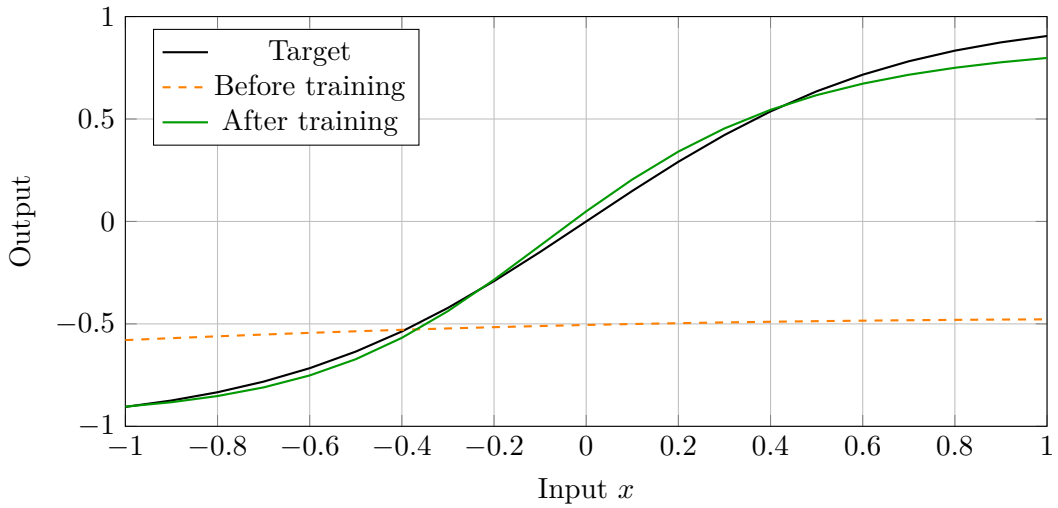


Figure 3: Targets and model predictions before and after training.

Table 3: Selected predictions, rounded to three decimals.

Input x	Target	Before	After
−1.0	−0.905	−0.579	−0.904
−0.5	−0.635	−0.536	−0.672
0.0	0.000	−0.505	0.049
0.5	0.635	−0.487	0.617
1.0	0.905	−0.478	0.798

9 Conceptual Discussion

Why use topological order? In a shared graph, a node can receive gradients from several later operations. Reverse topological order lets those operations send their contributions before the node's backward function runs.

Why accumulate gradients? For $d = (a + a)a$ at $a = 3$, multiplication contributes 6 directly to a, and addition contributes $3 + 3$. Assignment would lose contributions; accumulation gives the correct 12.

Why reset gradients? Old gradients remain stored in parameters. Calling `zero_grad()` each iteration ensures an update uses only the current loss.

10 Limitations and Conclusion

A scalar engine creates many Python objects and does not offer the efficiency of tensor-based systems. The experiment uses a small generated dataset and does not measure generalization. The final neuron uses `tanh`, so its output is bounded between -1 and 1.

Within this scope, all 12 course tests passed, analytical and numerical gradient checks agreed, and the specified MLP reduced its training loss. These observations support the implemented graph construction and backpropagation behavior.